Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет **МДК.03.02**

Тема: **Комплексное тестирование ПО и отчёт о тестировании**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ОБЪЕКТ ТЕСТИРОВАНИЯ И ПЛАН.....4
2 РЕАЛИЗОВАННЫЕ ТЕСТЫ И РЕЗУЛЬТАТЫ.....6
3 АНАЛИЗ ДЕФЕКТОВ И ИТОГОВЫЙ ОТЧЁТ.....8
ЗАКЛЮЧЕНИЕ.....9
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....10

ВВЕДЕНИЕ

Цель работы: закрепить навыки комплексного тестирования микросервиса (модульные и интеграционные тесты, проверка валидации, изоляция данных) и составить отчёт о тестировании по требованиям ISO/IEC 25010 [1].

Работа выполнена по варианту 3: модуль «Умный климат-контроль (Climate)» АИС «Умный кампус». Студент выступает в роли QA-инженера и должен доказать, что модуль готов к развёртыванию.

1 ОБЪЕКТ ТЕСТИРОВАНИЯ И ПЛАН

1.1 Объект тестирования

Тестируется Web API модуля Climate: сущность ClimateSensorRecord (код аудитории, температура, влажность) и контроллер с эндпоинтами POST /api/v1/climate/sensor (сохранить показание) и GET /api/v1/climate/sensor (получить список). Поле AuditoriumCode обязательно ([Required]) [2]; при неверных данных атрибут [ApiController] автоматически возвращает 400 Bad Request [3].

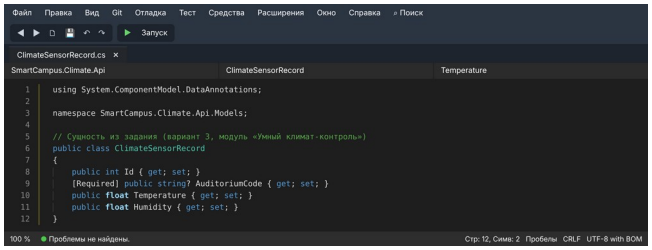


Рисунок 1 – Сущность ClimateSensorRecord

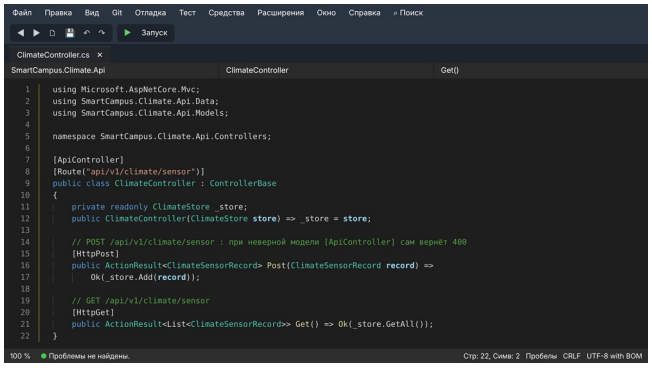


Рисунок 2 – Контроллер ClimateController

1.2 План тестирования

Таблица 1 – План тестирования

Тест	**Вид**	**Характеристика ISO/IEC 25010**	**Ожидаемый результат**
Тест 1	Позитивный (функциональный)	Functional suitability	POST возвращает 200 OK, запись видна в GET
Тест 2	Негативный (валидация)	Reliability, Security (проверка входных данных)	POST с AuditoriumCode = null возвращает 400, запись не сохраняется
Тест 3	Интеграционный (изоляция)	Reliability (независимость тестов)	Каждый тест начинается с пустой базы

Интеграционные тесты запускают API целиком и обращаются к нему по HTTP, как реальный клиент [4]. Для изоляции создан класс CustomApiFactory:

он поднимает отдельный экземпляр приложения со своим пустым хранилищем на свободном порту и после теста останавливает его. Это аналог CustomWebApplicationFactory.

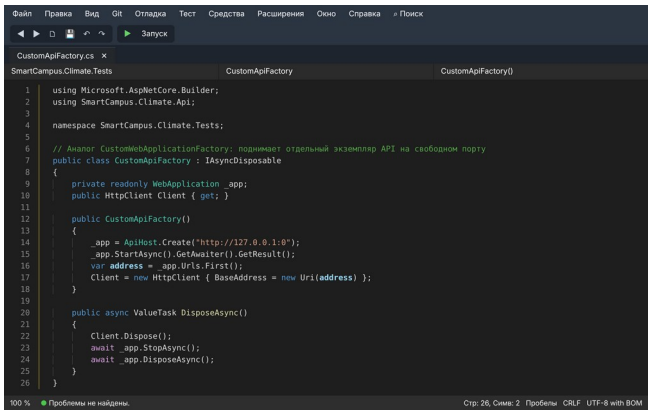


Рисунок 3 – Класс CustomApiFactory

2 РЕАЛИЗОВАННЫЕ ТЕСТЫ И РЕЗУЛЬТАТЫ

Тест 1 создаёт запись и проверяет код 200 и то, что GET возвращает ту же запись. Тест 2 передаёт AuditoriumCode = null и ожидает 400, а также проверяет, что запись не сохранилась. Тест 3 сохраняет запись в одном экземпляре приложения, затем создаёт новый и проверяет, что список пуст.

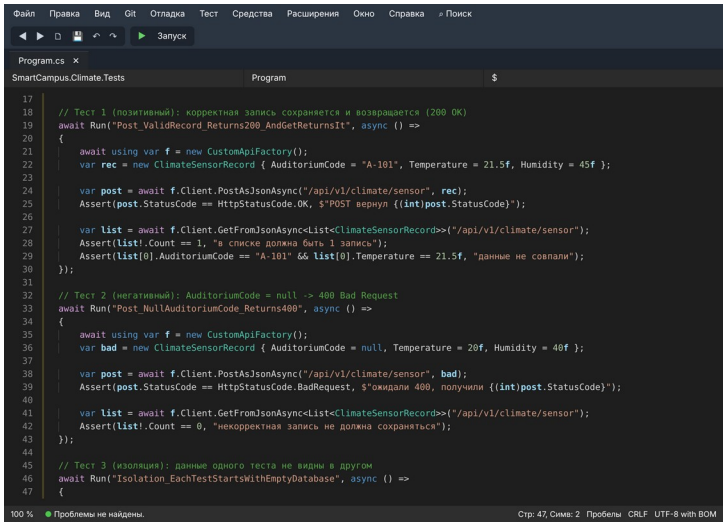


Рисунок 4 – Тесты 1 и 2

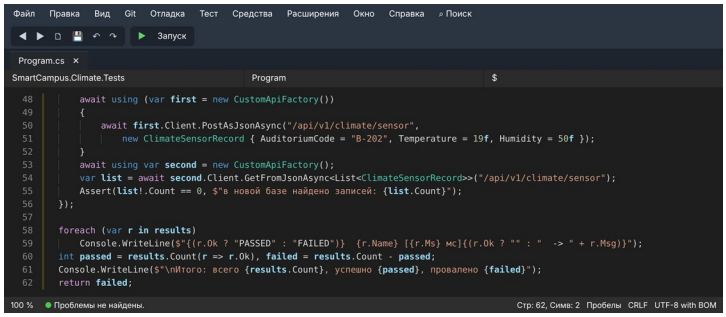


Рисунок 5 – Тест 3 (изоляция) и вывод результатов

В учебной среде тесты оформлены как консольный запуск с собственной проверкой Assert, а хранилище реализовано в памяти (ClimateStore). При работе в Visual Studio с пакетами NuGet их можно перенести в xUnit [5] и заменить хранилище на SQLite In-Memory, логика тестов при этом не меняется.

Результаты выполнения

Все три теста пройдены: 3 из 3 (рисунки 6 и 7).



Рисунок 6 – Обозреватель тестов: 3 из 3 тестов пройдены

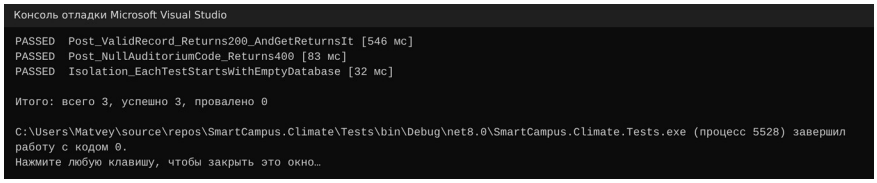


Рисунок 7 – Консоль отладки: результат запуска

3 АНАЛИЗ ДЕФЕКТОВ И ИТОГОВЫЙ ОТЧЁТ

3.1 Анализ дефектов и рисков

При первом запуске все три теста упали с кодом 404 Not Found. Это пример связи первичной ошибки и вторичных симптомов.

Таблица 2 – Анализ дефекта

Что наблюдали	**Причина**
Первичная ошибка	Приложение запускалось из сборки тестов, а контроллер находится в другой сборке, поэтому MVC его не нашёл и маршруты не зарегистрировались
Вторичные симптомы	Все три теста получили 404 вместо 200 и 400, хотя сама логика контроллера не менялась
Исправление	В ApiHost добавлен AddApplicationPart(typeof(ApiHost).Assembly), после этого все тесты пройдены

Остаточные риски: хранилище в памяти не проверяет работу с реальной БД; не проверены граничные значения температуры и влажности (в сущности нет ограничений Range); не проверена нагрузка. Для следующей версии рекомендуется добавить ограничения диапазонов и тесты на них.

3.2 Итоговый отчёт о тестировании

Таблица 3 – Отчёт о тестировании программного продукта [6]

Пункт	**Содержание**
1. ФИО студента	Браунков Матвей Константинович
2. Вариант и модуль АИС	Вариант 3, Умный климат-контроль (Climate)
3. Цель тестирования	Проверка функциональной пригодности и надёжности API (ISO/IEC 25010)
4. Сводка результатов	Всего тест-кейсов: 3. Passed: 3. Failed: 0. Общий статус: ПЕРЕДАН
5. Реализованные тесты	Тест 1 (позитивный): Post_ValidRecord_Returns200_AndGetReturnsIt – сохранение корректных данных. Тест 2 (негативный): Post_NullAuditoriumCode_Returns400 – возврат 400 Bad Request. Тест 3 (изоляция): Isolation_EachTestStartsWithEmptyDatabase – чистота базы между тестами
6. Скриншот	Рисунок 6 (обозреватель тестов) и рисунок 7 (консоль)
7. Заключение	Модуль соответствует требованиям функциональной пригодности и надёжности и готов к интеграции с учётом остаточных рисков из раздела 3

ЗАКЛЮЧЕНИЕ

Для модуля Climate написаны и выполнены три теста: позитивный, негативный (валидация, 400 Bad Request) и интеграционный (изоляция данных). Все тесты пройдены, итоговый вердикт – модуль готов к развёртыванию.

В ходе работы найден и устранён дефект конфигурации (404 из-за незарегистрированного контроллера), а также описаны остаточные риски и рекомендации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ISO/IEC 25010:2011. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models [Электронный ресурс]. – URL: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (дата обращения: 30.10.2026).

2. Microsoft Learn. Model validation in ASP.NET Core MVC [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation (дата обращения: 30.10.2026).

3. Microsoft Learn. Create web APIs with ASP.NET Core: ApiController attribute [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/aspnet/core/web-api/ (дата обращения: 30.10.2026).

4. Microsoft Learn. Integration tests in ASP.NET Core [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests (дата обращения: 30.10.2026).

5. xUnit.net. Getting started: .NET Core with command line [Электронный ресурс]. – URL: https://xunit.net/docs/getting-started/netcore/cmdline (дата обращения: 30.10.2026).

6. IEEE 829-2008. Standard for Software and System Test Documentation [Электронный ресурс]. – URL: https://standards.ieee.org/ieee/829/3787/ (дата обращения: 30.10.2026).